

Employee Payroll System

Enterprise Architecture, Implementation Details & Interview Technical Guide

1. Executive Project Overview

The **Employee Payroll Management System** is a production-ready, RESTful enterprise microservice backend built with **Spring Boot 3.2.5** and **Java 17**. It handles employee lifecycle data, corporate department management, salary tracking, multi-criteria dynamic searching, pagination, automated audit logging, and interactive OpenAPI/Swagger documentation.

Component	Technology / Framework	Purpose
Framework	Spring Boot 3.2.5	Core backend microservice architecture
Language	Java 17 (LTS)	Modern Java platform & features
Data Access	Spring Data JPA & Hibernate	ORM mapping and repository pattern
Database	H2 In-Memory DB	Embedded relational database engine
Cross-Cutting	Spring AOP (AspectJ)	Centralized audit logging & execution metrics
Validation	Jakarta Bean Validation	Declarative constraints (@NotBlank, @Email, etc.)
Documentation	SpringDoc OpenAPI v2.5.0	Interactive Swagger UI & API specs
Testing	JUnit 5 & MockMvc	Automated integration & endpoint testing

2. Layered Architecture (N-Tier Architecture)

The application strictly adheres to the **Layered Architecture Pattern**, ensuring separation of concerns, maintainability, testability, and clean dependency management.

Client / Front-End / Postman

▼ HTTP REST Requests (JSON)

1. Controller Layer (@RestController): Handles HTTP routes, payload validation (@Valid), and HTTP responses.

▼ DTO Objects (EmployeeDTO, DepartmentDTO, PageResponseDTO)

2. Service Layer (@Service): Encapsulates business logic, email rules, entity conversions, QBE matching.

▼ JPA Entities (Employee, Department)

3. Repository Layer (@Repository): Extends JpaRepository, handles DB queries via Spring Data JPA.

▼ SQL / JDBC

4. Database Layer (H2 DB): Embedded relational database storing tables ('employees', 'departments').

** Cross-Cutting Aspect: Spring AOP (@Aspect) intercepts Controller, Service, & Repository for audit logs.*

Layer Responsibilities Breakdown:

- **Controller Layer (com.cjc.payroll.controller):** Exposes RESTful API endpoints (/api/employees, /api/departments). Uses DTOs to isolate API contracts from DB entities. Annotated with OpenAPI Swagger documentation.

- **Service Layer (com.cjc.payroll.service):** Contains core business logic (EmployeeServiceImpl, DepartmentServiceImpl). Enforces business rules (e.g., prohibiting @gmail.com corporate email usage, email uniqueness checks).
- **Repository Layer (com.cjc.payroll.repository):** Spring Data JPA interfaces (EmployeeRepository, DepartmentRepository). Eliminates boilerplate DAO implementation code.
- **DTO & Model Layer (com.cjc.payroll.dto / entity):** Decouples database schema (Employee, Department) from external REST representations (EmployeeDTO, DepartmentDTO, PageResponseDTO, ErrorResponse).
- **Exception & Aspect Layer (com.cjc.payroll.exception / aspect):** Centralized exception handling via @RestControllerAdvice and automated performance logging via Spring AOP.

3. Core Technical Features Implemented

- **Full CRUD Operations:** Complete REST API support for Creating, Reading, Updating, and Deleting employees and departments with standard HTTP status codes (201 Created, 200 OK, 400 Bad Request, 404 Not Found).
- **Dynamic Pagination & Sorting:** Implemented using Spring Data Pageable, PageRequest, and Sort. Returns metadata encapsulated inside a standardized generic PageResponseDTO (totalElements, totalPages, pageNo, pageSize, isLast).
- **Query By Example (QBE) Search:** Flexible multi-criteria dynamic search without custom JPQL/SQL. Built with ExampleMatcher (matching nulls, ignoring case, string containing matcher).
- **Corporate Email Rule & Business Validations:** Enforces corporate email usage (rejects personal @gmail.com domain via Regex @Pattern and Java logic) and checks duplicate email existence.
- **Centralized Global Exception Handling:** @RestControllerAdvice catches ResourceNotFoundException, MethodArgumentNotValidException, and IllegalArgumentException, returning consistent structured JSON ErrorResponse objects.
- **Aspect-Oriented Audit & Performance Logging:** LoggingAspect uses @Around and @AfterThrowing pointcuts across Controller, Service, and Repository layers to measure method execution time in ms and log execution entry/exit/exceptions.
- **OpenAPI / Swagger 3 Interactive Docs:** SpringDoc OpenAPI UI integration with custom @Tag, @Operation, @ApiResponse, and @Schema annotations for automated documentation generation at /swagger-ui.html.
- **Automated Integration Testing:** Spring Boot MockMvc test suite (EmployeeControllerTest, DepartmentControllerTest) covering end-to-end CRUD, validation rejections, and pagination.

4. Key REST API Endpoints

HTTP Method	Endpoint Path	Description	Status Code
POST	/api/employees	Create new employee with validation	201 Created
GET	/api/employees	Retrieve all employees (unpaginated)	200 OK
GET	/api/employees/{id}	Get employee details by ID	200 / 404
PUT	/api/employees/{id}	Update existing employee by ID	200 / 400 / 404
DELETE	/api/employees/{id}	Delete employee record by ID	200 / 404
GET	/api/employees/search	Dynamic search + pagination + sorting	200 OK
POST	/api/departments	Create new department	201 Created
GET	/api/departments/search	Paginated department search	200 OK

5. Interview Explanation Scripts & Top Q&A;

Script 1: 30-Second Project Pitch

"In my project, I developed an Employee Payroll backend REST microservice using Java 17 and Spring Boot 3. It features a clean 3-tier layered architecture (Controller, Service, Repository, DTO). Key highlights include dynamic multi-criteria searching & pagination using Spring Data Query-by-Example, centralized global exception handling with @RestControllerAdvice, aspect-oriented performance & audit logging using Spring AOP, input validation for business rules like enforcing corporate email constraints, and interactive Swagger/OpenAPI documentation."

Q1. Why did you use DTOs instead of passing Entities directly to Controllers?

Entities represent internal database tables. Direct exposure leaks DB schema details and risks circular reference errors during JSON serialization. DTOs decouple API contracts from database schema, enforce payload validation using Jakarta annotations, and keep security/privacy intact.

Q2. How did you implement Dynamic Search and Pagination?

I combined Spring Data JPA's Pageable API with ExampleMatcher (Query-by-Example). The search controller accepts query parameters (pageNo, pageSize, sortBy, sortDir, name, dept, email). The Service layer constructs an Example probe with string matching rules (CONTAINING, IGNORE_CASE) and returns a PageResponseDTO containing pagination metadata.

Q3. How is Global Exception Handling implemented?

I created a @RestControllerAdvice class (GlobalExceptionHandler) with @ExceptionHandler methods for ResourceNotFoundException (404), MethodArgumentNotValidException (400 validation error map), and IllegalArgumentException (400 bad request), returning consistent ErrorResponse JSON objects.

Q4. What is the role of AOP in your application?

Spring AOP separates cross-cutting concerns like logging from core business logic. LoggingAspect uses @Around advice to intercept Controller, Service, and Repository execution, measuring execution time in milliseconds and logging entry/exit and exceptions automatically.

Q5. How do you enforce business validation rules?

Through declarative Jakarta Bean Validation annotations (@NotBlank, @Email, @Pattern, @Positive) in DTOs and custom service logic in EmployeeServiceImpl that checks duplicate emails and prohibits personal @gmail.com domains.

6. Key Takeaways for Interviewers

- **Architecture:** Standardized Spring Boot 3-Tier Layered Architecture with DTO Pattern.
- **Clean Code & SOLID:** Single Responsibility Principle across layers, Open/Closed with interfaces.
- **Production Quality:** Centralized error handling, AOP performance logging, automated integration testing, OpenAPI docs.